

Aula 5: A Camada de Dados e Alta Disponibilidade no SAGA

Fundamentos de arquitetura eficiente e sustentável para o ecossistema AgroVision
SAGA — processando e armazenando de forma resiliente os dados de **50.000 sensores de campo**, com foco em performance, custo e alta disponibilidade.

BLOCO 1 AO 5

WELL-ARCHITECTED FRAMEWORK



Contexto de Negócio: AgroVision SAGA

O ecossistema AgroVision SAGA precisa processar e armazenar de forma resiliente os dados vindos de **50.000 sensores de campo**. Esses dados se dividem em dois grandes grupos com características completamente distintas, exigindo soluções arquiteturais diferentes para cada um.



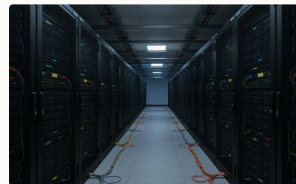
Imagens HD de Folhas

Grandes arquivos de mídia capturados pelos sensores para análise por Inteligência Artificial na detecção de pragas. Exigem armazenamento durável, escalável e de baixo custo para arquivamento de longo prazo.



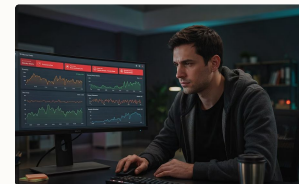
Telemetria de Umidade e Temperatura

Milhares de pequenas escritas por segundo vindas dos sensores de campo. Exigem latência mínima, alta taxa de transferência e escalabilidade horizontal sem degradação de performance.



Banco Relacional Corporativo

Dados cadastrais de produtores, contratos de cooperativas e faturamento precisam de transações ACID e **alta disponibilidade** com failover automático entre zonas de disponibilidade.



Alertas em Tempo Real

O time de engenharia precisa de **alertas automáticos** sobre a saúde da infraestrutura — CPU, rede, latência — sem depender de chamados reativos dos clientes.

O Dilema da Escala no Agro

Como arquitetos de soluções, o maior desafio ao lidar com 50.000 sensores não é apenas fazer o sistema funcionar, mas garantir que ele seja **economicamente viável para a AgroVision** e rápido para o produtor rural na ponta.

O Perigo do Superprovisionamento

Na computação tradicional (on-premises), a TI precisava comprar servidores calculando o **pior cenário possível de tráfego** — o pico do ano. Se a AgroVision fizesse isso na nuvem, comprando uma instância RDS ou EC2 gigante para aguentar as milhares de escritas dos sensores durante o dia, violaria diretamente o pilar de **Otimização de Custos** do Well-Architected Framework.

A Mentalidade de Nuvem

Durante a noite ou períodos de entressafra, servidores superprovisionados ficam **ociosos e subutilizados**, queimando o caixa da startup à toa. A mentalidade correta de nuvem exige projetar sistemas que consumam recursos de forma **proporcional à demanda real** — pagando apenas pelo que é efetivamente utilizado, no momento em que é utilizado.

- ❑ Overprovisioning na nuvem é tão prejudicial quanto subutilização em hardware físico — ambos representam desperdício financeiro evitável.

Eficiência de Performance

O pilar de **Eficiência de Performance** foca na capacidade de usar os recursos computacionais de forma eficiente para atender aos requisitos do sistema. No ecossistema SAGA, aplicamos duas grandes diretrizes deste pilar.

1. Seleção Orientada ao Padrão de Acesso (Data Access Patterns)

O Framework deixa claro que um arquiteto de sucesso escolhe a tecnologia com base em **como os dados são gravados e lidos**. O banco tradicional RDS é excelente para relacionar tabelas e contratos complexos. Porém, se 50.000 sensores tentarem escrever dados simples de umidade simultaneamente, o banco sofrerá com **exaustão de conexões e travas de tabelas**, degradando a performance do sistema inteiro.

Para a telemetria em alta frequência, mudamos o paradigma para o **Amazon DynamoDB**. Ele gerencia clusters internos que distribuem os dados horizontalmente com base em uma **Chave de Partição (sensor_id)**. O resultado prático é que a latência para gravar ou ler a umidade do solo permanece estável em **um dígito de milissegundo**, seja com 10 sensores ou 50.000.

2. Elasticidade e Dimensionamento Dinâmico

Em vez de tentar prever o hardware exato, a nuvem oferece a agilidade de **alterar o tamanho de instâncias e volumes de armazenamento (IOPS) "a quente"**. Se detectarmos um gargalo de performance no RDS corporativo, podemos mudar o tipo de instância imediatamente de dentro da plataforma para atender à nova demanda — sem janelas de manutenção ou interrupções de serviço.

Otimização de Custos

O pilar de **Otimização de Custos** nos desafia a refinar continuamente nossa arquitetura para eliminar desperdícios financeiros. Não basta a infraestrutura ser rápida; ela precisa ter o **menor custo residual possível**.

1. Modelos de Capacidade Financeira no DynamoDB

O arquiteto precisa tomar uma decisão financeira crucial baseada no comportamento do negócio:

- **Provisioned (Capacidade Fixa):** Você paga por uma velocidade contratada, use-a ou não. Ideal para tráfego previsível e constante.
- **On-Demand (Sob Demanda):** Você paga exatamente por cada leitura ou escrita efetuada. Para o SAGA, onde os sensores transmitem muito durante o sol a pino no campo e quase nada na calada da noite, o modelo On-Demand **evita o desperdício de pagar por capacidade ociosa**.

2. Gerenciamento do Ciclo de Vida dos Dados (Lifecycle Policies)

O armazenamento no Amazon S3 é incrivelmente durável (**11 noves de durabilidade**) e ideal para guardar as fotos das plantações. No entanto, manter imagens antigas de safras passadas na classe S3 Standard é um **erro grave de custo**.

Aplicamos o conceito de **Hierarquização de Armazenamento (Storage Tiering)**: criamos regras automáticas que movem arquivos antigos para classes de arquivamento frio, como o **Amazon S3 Glacier**. O dado permanece seguro e disponível para auditorias, mas o custo por gigabyte armazenado **despenca em até 90%** de forma 100% automatizada.

Bloco 2: Data Lake de Imagens com Amazon S3

APANHADO TEÓRICO

Os sensores capturam imagens de folhas em alta definição para que a IA detecte pragas. Salvar essas imagens dentro do disco rígido (EBS) das instâncias EC2 gera três problemas graves: **custo alto por gigabyte**, **limite de tamanho físico dos discos** e, se a instância EC2 for destruída pelo Auto Scaling, **as fotos dos clientes somem**.

Amazon S3: Armazenamento de Objetos

O Simple Storage Service é um serviço de armazenamento de **objetos** (arquivos), não de blocos. Projetado para oferecer **99.999999999% (11 noves) de durabilidade** — estatisticamente, você pode perder um objeto a cada 10.000 anos ao armazenar 10.000 objetos.

Imutabilidade e Buckets

Os objetos são **imutáveis**: alterar 1 byte de uma foto gera uma nova versão ou sobrescreve o objeto por inteiro. Os arquivos ficam dentro de contêineres lógicos chamados **Buckets**, cujo nome deve ser **globalmente exclusivo** em toda a AWS.

Lifecycle Policies

Imagens tiradas há 1 ano raramente são consultadas, mas precisam ser guardadas por questões regulatórias. Regras de ciclo de vida movem automaticamente esses arquivos do **S3 Standard** para o **S3 Glacier Flexible Retrieval** ou **Glacier Deep Archive** — até **90% mais barato** — após um período de dias determinado.

Passo a Passo: Criação do Bucket e Upload

01

Criar o Bucket

Acesse S3 no console (região **us-east-1**). Clique em **Create bucket**. Defina o nome globalmente exclusivo (ex: *agrovision-datalake-fotos-celio2026*). Mantenha **Block all public access** marcado. Habilite **Bucket Versioning** para proteger contra exclusões acidentais.

02

Configurar Criptografia SSE-KMS

Em **Default encryption**, selecione *Server-side encryption with AWS KMS keys (SSE-KMS)*. Escolha a chave customizada criada na aula de segurança (*kms-agrovision-saga*). Clique em **Create bucket**.

03

Upload e Validação

Clique no bucket criado, depois em **Upload** → **Add files**. Escolha qualquer imagem (simulando a foto do sensor de pragas). Após o upload, abra os detalhes do arquivo e valide que o campo **Server-side encryption settings** exibe o status criptografado com a chave KMS. Teste o **Download** para simular o aplicativo do fazendeiro recuperando a foto.

Bloco 3: Banco Relacional com Amazon RDS Multi-AZ

ALTA DISPONIBILIDADE

Por que RDS Gerenciado?

As informações estruturadas da AgroVision (dados cadastrais de produtores, contratos de cooperativas e faturamento) precisam de **transações ACID puras** em MySQL ou PostgreSQL. Gerenciar isso em uma EC2 comum exige que o arquiteto cuide de atualizações do SO, rotinas de backups manuais e arquiteturas complexas de replicação. O **Amazon RDS** elimina esse overhead: a AWS cuida do provisionamento, patches e backups automáticos.

Segurança e Isolamento

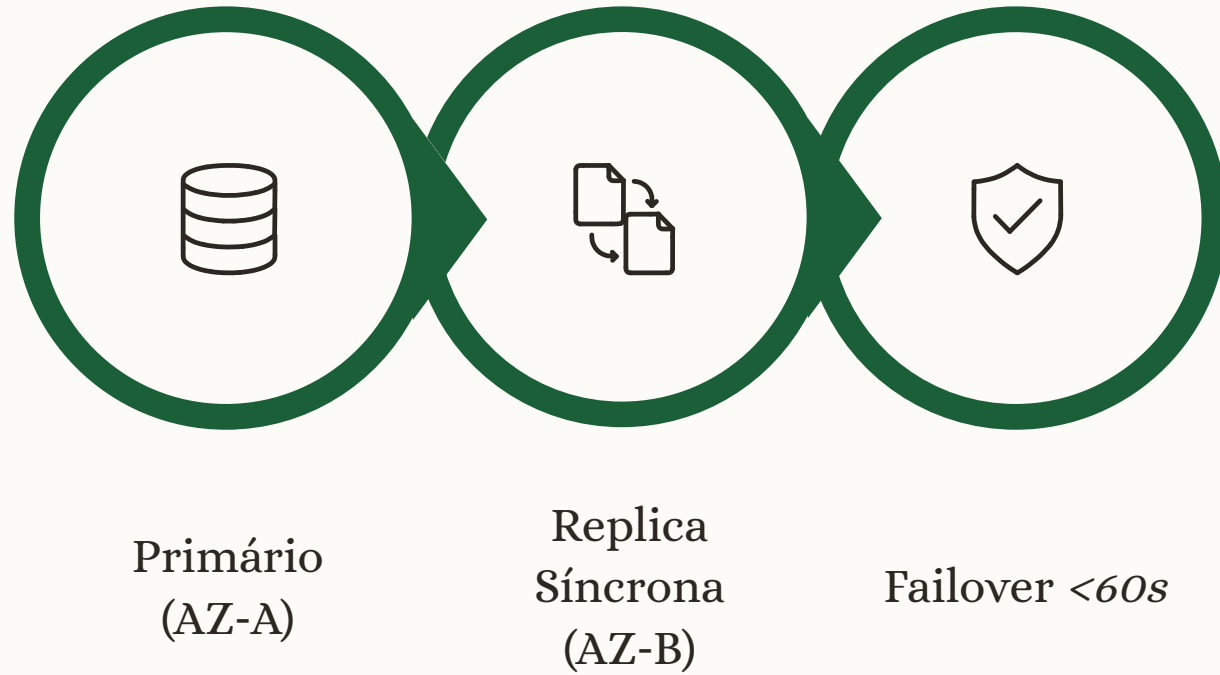
O RDS deve sempre morar em **Subnets Privadas**, sem IP público. Apenas as instâncias EC2 da camada web possuem caminhos de rede (via Security Groups) para conversar com ele.

Alta Disponibilidade com Multi-AZ

Quando ativamos a opção **Multi-AZ**, a AWS cria uma instância **Principal (Primary)** na Subnet Privada da AZ-A e provisiona automaticamente uma instância **Réplica (Standby) síncrona** na Subnet Privada da AZ-B.

Tudo o que é gravado na AZ-A é replicado imediatamente a nível de blocos na AZ-B. Se a AZ-A sofrer uma pane física catastrófica, o RDS altera o apontamento do DNS interno de forma automatizada (**Failover**), e a réplica da AZ-B assume a liderança em **menos de 60 segundos**, sem que você precise alterar uma linha de código nas instâncias EC2.

- ✓ Failover automático em menos de 60 segundos — sem alteração de código nas aplicações.



O modelo Multi-AZ garante que a persistência dos dados corporativos da AgroVision seja blindada contra falhas físicas regionais, sem intervenção manual da equipe de engenharia.

Passo a Passo: Provisionando o RDS

01

Criar o Banco de Dados

Acesse **RDS → Databases → Create database**. Escolha **Standard create**. Em Engine options, selecione **SQLServer, PostgreSQL ou MySQL**. Em Templates, selecione **Production** (habilita opções reais de Multi-AZ). Configure: *DB instance identifier: agrovision-saga-db, Master username: admin*, senha forte (ex: AgroVision2026!). Em Instance configuration, use classes menores para o laboratório (ex: **db.t3.micro**).

02

Ativar Multi-AZ e Conectividade

Em **Availability & durability**, selecione *Create a standby instance (recommended for production usage)* — esta é a chave que ativa o modelo Multi-AZ. Em Connectivity: selecione a **AgroVision-Manual-VPC**, defina **Public access: No** e crie um novo Security Group chamado *sg-rds-agrovision*. Clique em **Create database** (provisionamento leva 5–10 minutos, pois a AWS configura fisicamente duas zonas distintas).

03

Dimensionamento "a Quente" e Migração

Enquanto o banco inicializa, demonstre como alterar o hardware de um banco corporativo sem downtime: clique em **Modify**, altere o tipo de instância ou aumente IOPS, e use **Apply immediately**. Para migração de dados legados, apresente o **AWS DMS (Database Migration Service)**: ele lê o banco on-premises (Source Endpoint), replica continuamente via rede e injeta os dados no novo RDS (Target Endpoint) de forma assíncrona, sem desligar o sistema da cooperativa.

Bloco 4: Telemetria Massiva com Amazon DynamoDB

NOSQL · SERVERLESS

Com **50.000 sensores gravando pequenas linhas de telemetria de umidade a cada segundo**, direcionar todas essas conexões simultâneas de escrita direta para o banco relacional RDS causaria **exaustão de conexões, deadlocks e esgotamento de IOPS** — derrubando o sistema corporativo por completo.

DynamoDB: NoSQL Serverless

Banco de dados chave-valor **totalmente gerenciado**. Sem servidores para gerenciar, sem VPC complexa. Entrega **latência consistente de um dígito de milissegundo** em qualquer escala de tráfego.

Escalabilidade Horizontal

Diferente do RDS (escala vertical), o DynamoDB divide os dados horizontalmente via **Partition Key** (**sensor_id**). Uma função de hash interna determina em qual servidor físico da AWS aquele dado será escrito de forma instantânea.

Provisioned vs. On-Demand

Provisioned: você define RCUs e WCUs por segundo — ideal para tráfego previsível. **On-Demand:** a AWS escala leituras e escritas instantaneamente conforme o volume que chega — ideal para o SAGA, com picos diurnos e baixo tráfego noturno.

Passo a Passo: Criando a Tabela DynamoDB

01

Criar a Tabela NoSQL

Acesse **DynamoDB** → **Create table**. Configure:
Table name: AgroVision-Telemetria-IoT.

Partition key: *sensor_id* (tipo String) — distribuirá a telemetria uniformemente pelos clusters da AWS. **Sort key:** *timestamp* (tipo Number) — permite ordenar e buscar o histórico de leituras de um mesmo sensor por ordem cronológica. Em Table settings, desmarque Default settings e selecione **Customize settings**.

02

Selecionar Modo On-Demand

Em **Read/write capacity mode**, selecione *On-demand (Sob Demanda)*. Isso garante **escalabilidade infinita automática** sem que seja necessário calcular capacidade no primeiro dia. Deixe as demais opções de criptografia padrão ativas e clique em **Create table**.

03

Inserir Dados e Demonstrar Flexibilidade NoSQL

Após o status mudar para **Active**, clique na tabela → **Actions** → **Explore items** → **Create item**. Preencha: *sensor_id: SENSOR-FAZENDA-CENTRO-01*, *timestamp: 1787162400*, *umidade_solo: 42* (Number), *status_bateria: OK* (String). Crie um segundo item para outro sensor com atributos totalmente diferentes (*temperatura_ar*, *coordenadas_gps*), demonstrando a vantagem do NoSQL **schemaless** — sem esquema engessado como no RDS.

Bloco 5: Monitoramento Avançado com Amazon CloudWatch

EXCELÊNCIA OPERACIONAL

Por que Monitorar Proativamente?

Para garantir a confiabilidade da AgroVision SAGA, os arquitetos **não podem esperar que o cliente ligue reclamando** que o sistema está fora do ar ou lento. Precisamos de telemetria interna da nossa própria infraestrutura — este é o pilar de **Excelência Operacional** do Well-Architected Framework.

O que é o Amazon CloudWatch?

É o **repositório central de monitoramento, métricas, logs e alarmes** da AWS. Serviços como instâncias EC2 enviam métricas automaticamente a cada 5 minutos (ou 1 minuto com monitoramento detalhado), capturando **Utilização de CPU, Tráfego de Rede e Utilização de Créditos de CPU**.

Alarmes Estatísticos e Ações Automatizadas

Um alarme monitora uma única métrica ao longo de um período definido. Podemos configurar **ações automatizadas** disparadas por esse alarme:

- Enviar notificação para o time de engenharia por **e-mail/SMS** usando o **Amazon SNS (Simple Notification Service)**
- Acionar uma política de **Auto Scaling** para criar novos servidores e absorver o impacto automaticamente

☑ Alarmes bem configurados transformam a operação de reativa para proativa — o time age antes que o produtor rural perceba qualquer degradação.

Passo a Passo: Criando o Alarme de Estresse de CPU

01

Selecionar a Métrica

Acesse **CloudWatch** → **Alarms** → **All alarms** → **Create alarm** → **Select metric**. Clique em **EC2** → **Per-Instance Metrics**. Localize a instância *AgroVision-Server-01-AZA* e selecione a métrica **CPUUtilization**. Clique em **Select metric**.

03

Configurar Ações e Notificação SNS

Em **Configure actions**: Alarm state trigger → *In alarm*. Selecione **Create new topic**: *Alerta-Engenharia-AgroVision*. Digite o e-mail corporativo ou de teste. Clique em **Create topic** (a AWS enviará um e-mail de confirmação — clique no link para confirmar a assinatura). Clique em **Next**.

02

Configurar Condições do Alarme

Em **Specify metric and conditions**: altere o **Period** de 5 minutos para *1 minute* (torna o alarme mais sensível a variações rápidas de tráfego dos sensores). Defina **Threshold type**: **Static**. Configure *CPUUtilization* **>= 80** (representando 80% de uso de CPU). Clique em **Next**.

04

Nomear e Finalizar

Defina o **Alarm name**: *Alarme-Estresse-CPU-Web01*. Clique em **Next**, revise as configurações no sumário e clique em **Create alarm**. O alarme passará a monitorar a CPU da instância e disparará automaticamente o e-mail para o time de engenharia quando o limiar de 80% for atingido ou superado.

Resumo: Pilares do Well-Architected Framework Observados

Ao longo desta aula, cada decisão arquitetural foi fundamentada em pilares concretos do **Well-Architected Framework**. Veja como os componentes implementados se conectam aos princípios que sustentam uma arquitetura eficiente, resiliente e economicamente viável.

1

Confiabilidade e Alta Disponibilidade

Ao mover o banco de dados para um **RDS Multi-AZ isolado em subnets privadas** e os arquivos de mídia para o **S3** (11 nozes de durabilidade), blindamos a persistência de dados contra falhas físicas regionais. O failover automático em menos de 60 segundos garante continuidade operacional sem intervenção humana.

2

Eficiência de Performance e Custos

O **DynamoDB On-Demand** absorve a escrita massiva de 50.000 sensores de campo com latência de um dígito de milissegundo, sem os custos astronômicos de infraestrutura pesada. As **Lifecycle Policies do S3** reduzem automaticamente o custo de armazenamento de imagens históricas em até 90%, sem qualquer intervenção manual.

3

Excelência Operacional

O **Amazon CloudWatch** com alarmes de CPU e integração ao **SNS** transforma a operação de reativa para proativa. O time de engenharia recebe alertas automáticos antes que o produtor rural perceba qualquer degradação, cumprindo o princípio de *"antecipar falhas e responder a eventos"* do pilar de Excelência Operacional.